

Challenge 3. Protecting communications and applications

Security in Computer Networks

Alejandro Pérez Bueno

May 20, 2026

Table of Contents

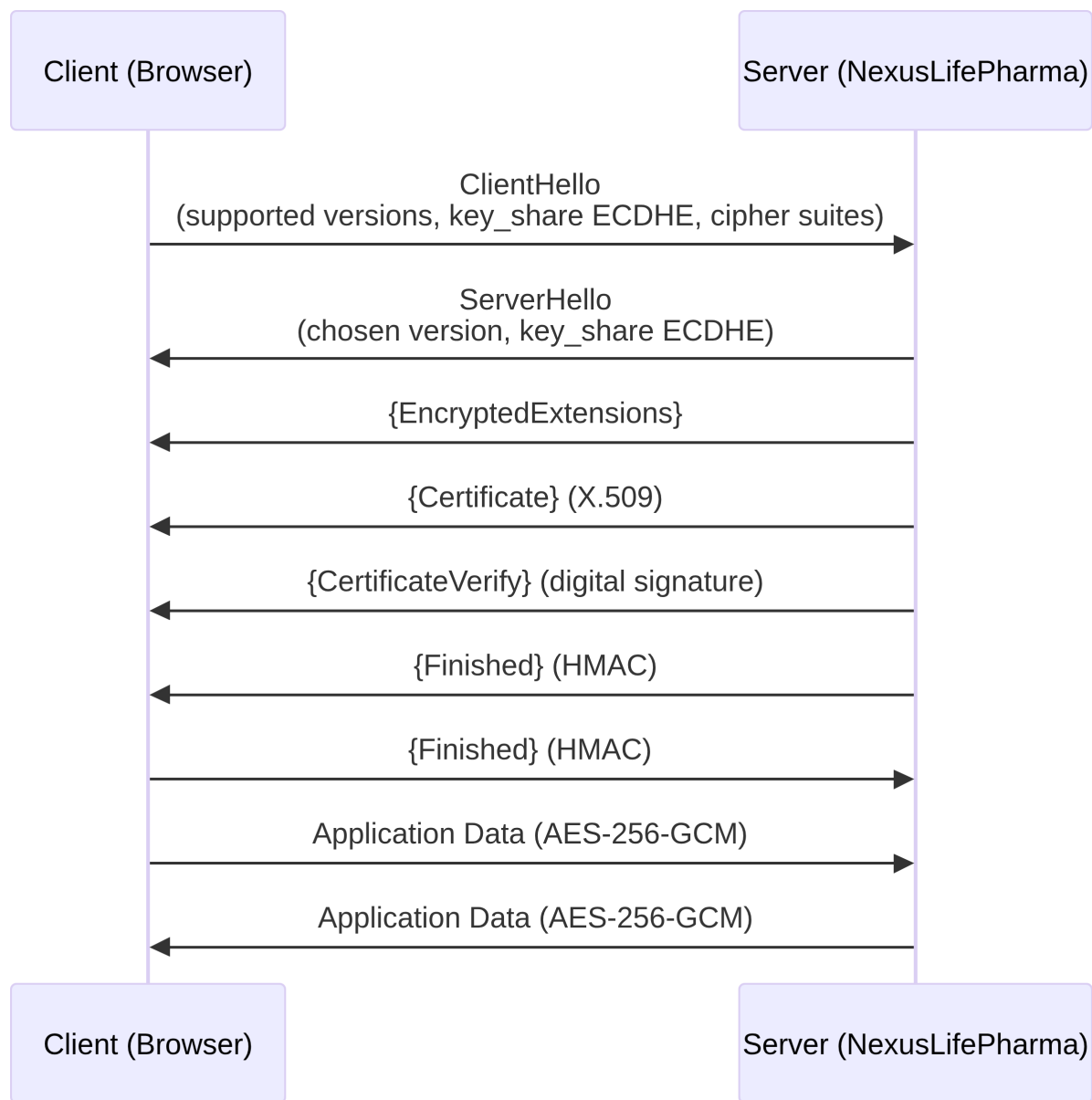
Activity 1	2
1.1	2
1.2	4
1.3	5
Activity 2	6
Activity 3	8
Final Reflection	9
References	10

Activity 1

1.1

NexusLifePharma is currently sending prescriptions, bank transactions and health data over plain HTTP. This means everything travels in plain text with no server authentication and no integrity check. TLS fixes this by setting up an encrypted channel before any application data gets exchanged ([Perramon Tornil 2023b](#)).

Below is a diagram of the TLS 1.3 handshake as defined in RFC 8446, which brings the negotiation down to a single round-trip ([Rescorla 2018](#)).

**i** Note

Messages in braces are encrypted.

After the **ServerHello** everything that follows including the server certificate is encrypted, so a passive observer cannot even see the server's identity ([Rescorla 2018](#); [Perramon Tornil 2023b](#)).

1.2

The table below shows the cryptographic primitives involved in each phase of the TLS handshake ([Perramon Tornil 2023a](#), [2023b](#)).

Phase	Cryptographic Primitives	Function
Authentication	Asymmetric Cryptography, Digital Signature with digital certificates (X.509)	The server sends its X.509 certificate signed by a trusted CA that binds its public key to its identity. The client verifies this signature to make sure it is talking to the real server.
Key Exchange	Diffie-Hellman or RSA to generate an ephemeral session key (Forward Secrecy)	Both sides exchange public DH values and each derives the same shared secret without ever transmitting it. Because computing discrete logarithms is hard an eavesdropper cannot recover the secret.
Data Cipherring	Symmetric Cryptography (AES-256)	Once the session keys are derived all application data gets encrypted with AES-256. Symmetric encryption is used here because it is much faster than asymmetric for bulk data.

Phase	Cryptographic Primitives	Function
Integrity	Hash Functions (HMAC / SHA-2)	Every TLS record includes an HMAC computed over a secret key, an implicit sequence number and the record contents. If anything changes in transit the HMAC will not match and the connection is dropped.

1.3

In older TLS setups that use static RSA key exchange the client encrypts a pre-master secret with the server's long-term RSA public key. So the security of every past session depends on that one private key staying secret ([Perramon Tornil 2023a](#)).

If an attacker records encrypted traffic and later gets hold of that private key (through theft, legal pressure or cryptanalysis) they can decrypt every session they previously captured. This is the “Harvest Now, Decrypt Later” threat model that we also discussed back in CAA 1 ([Perramon Tornil 2023a](#)).

Ephemeral key exchange solves this through Perfect Forward Secrecy (PFS). With Diffie-Hellman Ephemeral (DHE) or ECDHE both sides generate fresh key pairs per session and discard them right after. Since the shared secret cannot be reconstructed from the server's long-term key alone, compromising it later reveals nothing about past sessions ([Perramon Tornil 2023b](#), [2023a](#)).

TLS 1.3 takes care of this by requiring ephemeral key exchange in every cipher suite. Static RSA key exchange was removed from the specification entirely ([Rescorla 2018](#)).

Activity 2

NexusLifePharma needs to pick a session management mechanism for its pharmacist mobile app. JSON Web Tokens (JWT) are stateless tokens defined in RFC 7519 that encode session data into a signed token held by the client ([Jones et al. 2015](#)). Server-side sessions on the other hand store the data on the server and only send a session ID to the client through a cookie ([Perramon Tornil 2023b](#)).

The table below compares both approaches ([Jones et al. 2015](#); [OWASP 2025b](#)).

Criteria	JWT tokens	Sessions (Cookies)
State storage	Stateless: the token lives on the client side and contains all session data (claims) in a signed JSON object. The server does not keep any state.	Stateful: the server stores session data in memory or a database and sends the client just a session ID via a cookie.
Main risk	Token theft via XSS or man-in-the-middle. Because the token is self-contained a stolen JWT gives full access until it expires. The server cannot invalidate it early without extra infrastructure.	Session hijacking through cookie theft (XSS, network sniffing). But the session can be invalidated immediately on the server by deleting the record.
Revocation	Difficult: since JWTs are stateless there is no server record to delete. Revoking a token before expiration means maintaining a blacklist which adds statefulness and kind of defeats the purpose.	Simple: the server deletes or invalidates the session record at any time cutting access for that session ID.

For a high-traffic environment like this we would go with JWT tokens. Because they are stateless the server does not need to hit a session store on every request, which reduces

latency and makes horizontal scaling simpler. Session-based approaches need either sticky sessions or a shared store like Redis which adds complexity ([Jones et al. 2015](#)). The main downside of JWTs (hard revocation) can be mitigated with short expiration times combined with refresh tokens.

Activity 3

Social Security numbers and IBANs have a limited and predictable format. In Spain Social Security numbers are 12-digit numbers and IBANs follow a fixed structure starting with a country code and check digits. Because of this the search space is small enough that an attacker can precompute hashes for every possible value ([Perramon Tornil 2023a](#)).

SHA-256 alone is not enough to protect against something like the Energía XXI breach ([La Vanguardia 2026](#)). SHA-256 is a fast hash function designed for efficiency so an attacker with modern hardware can compute billions of hashes per second. On top of that without salting the attacker can build a rainbow table mapping every possible Social Security number or IBAN to its SHA-256 hash. Once the database is exfiltrated reversing the hashes becomes a simple lookup ([Perramon Tornil 2023a](#)).

Salting fixes part of the problem by adding random bits to each value before hashing. This way identical inputs produce different outputs and precomputation attacks stop working because the attacker has to compute a separate hash for every combination of salt and candidate value ([Perramon Tornil 2023a](#)).

To go further we propose combining salting with a Hardware Security Module (HSM). An HSM is a physical device that stores cryptographic keys and does operations inside tamper-resistant hardware so the keys never leave the module ([OWASP 2025a](#)). The architecture would work like this:

1. When a new Social Security number or IBAN is stored the application generates a unique random salt for that record.
2. The application sends the salt and the plaintext value to the HSM which computes an HMAC using a secret key that lives inside the HSM. The result gets stored in the database together with the salt.
3. To verify a value later the application sends the salt and the candidate to the HSM which recomputes the HMAC and returns whether it matches.

Because the HMAC key never leaves the HSM even if an attacker exfiltrates the whole database including salts they cannot compute the HMAC offline. They would need physical access to the HSM itself ([OWASP 2025a](#)).

Final Reflection

This CAA showed me how cryptographic primitives that I knew separately actually chain together in TLS to solve authentication, key exchange, ciphering and integrity in one handshake. The Perfect Forward Secrecy part stuck with me the most, the idea that a single static key can retroactively expose years of recorded traffic is something I now think about differently. The Energía XXI case was also a good reminder that hashing is not the same as protection, and the HSM approach with salting is something practical enough that I could bring up at work at Telefónica.

References

- Jones, Michael B., John Bradley, and Nat Sakimura. 2015. *JSON Web Token (JWT)*. RFC 7519. Internet Engineering Task Force (IETF). <https://doi.org/10.17487/RFC7519>.
- La Vanguardia. 2026. “Energía XXI Notifica Un Ciberataque Que Compromete Datos de Contratos y DNIs de Sus Clientes.” <https://www.lavanguardia.com/economia/20260111/11428973/energia-xxi-notificaciberataque-compromete-datos-contratos-dnis-clientes.html>.
- OWASP. 2025a. “Cryptographic Storage Cheat Sheet.” Open Worldwide Application Security Project. https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html.
- OWASP. 2025b. “JSON Web Token for Java Cheat Sheet.” Open Worldwide Application Security Project. https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html.
- Perramon Tornil, Xavier. 2023a. *Fundamentals of Encryption and Authentication*. 1st ed. Fundació Universitat Oberta de Catalunya (FUOC).
- Perramon Tornil, Xavier. 2023b. *Security in the Communications Layers*. 1st ed. Fundació Universitat Oberta de Catalunya (FUOC).
- Rescorla, Eric. 2018. *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446. Internet Engineering Task Force (IETF). <https://doi.org/10.17487/RFC8446>.